

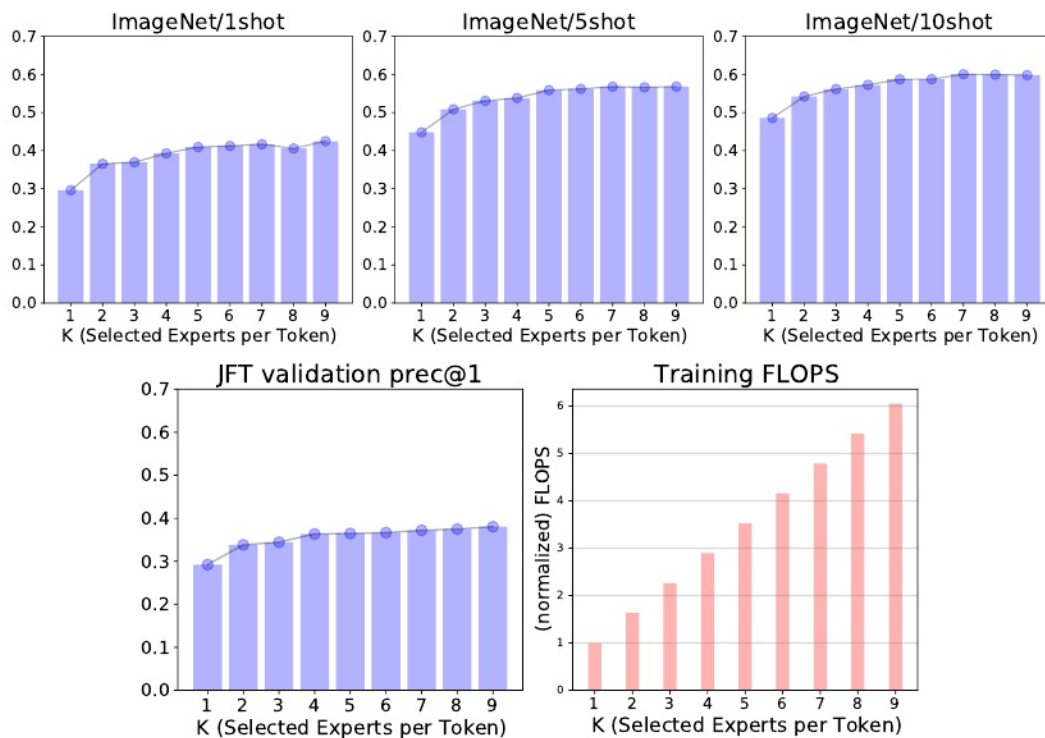


图10:上游, 少射和训练FLOPs作为k的函数对于每2 V-MoE-S/32

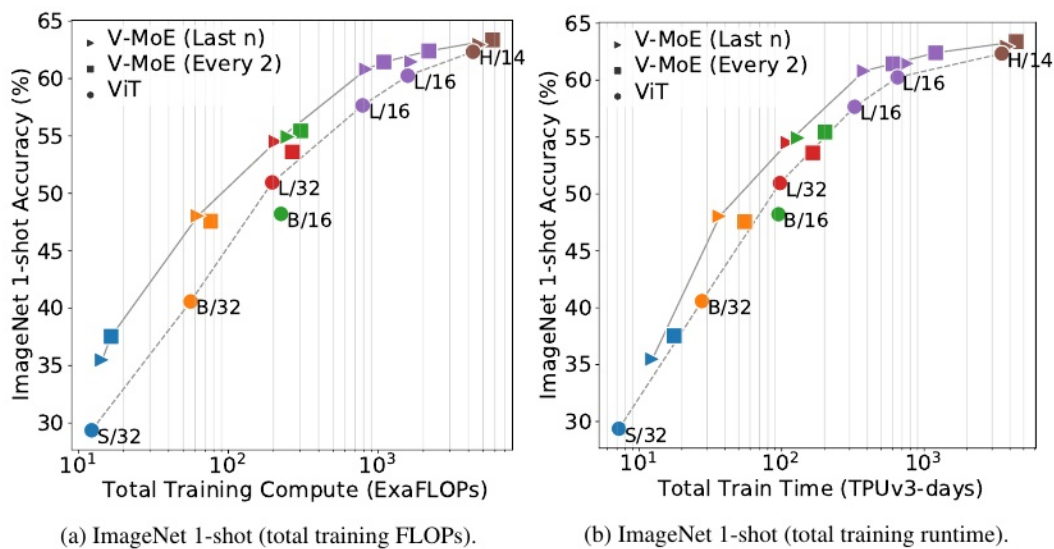


图11:稀疏和密集模型的ImageNet/1shot性能。(a)中的x轴表示训练期间所需的总FLOPs, 而(b)表示相同硬件的总训练时间。

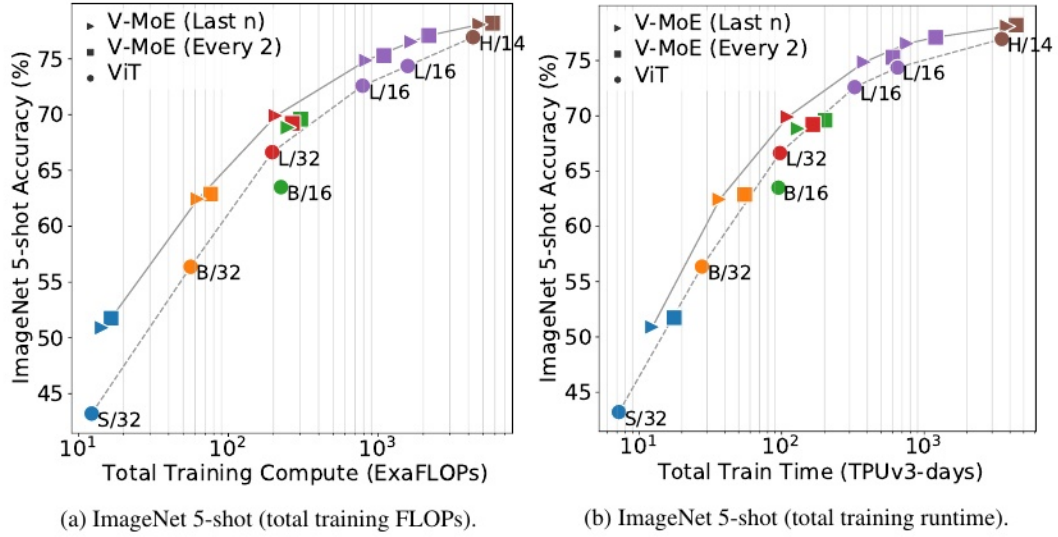


图12:稀疏和密集模型的ImageNet/5shot性能。(a)中的x轴表示训练期间所需的总FLOPs, 而(b)表示相同硬件的总训练时间。

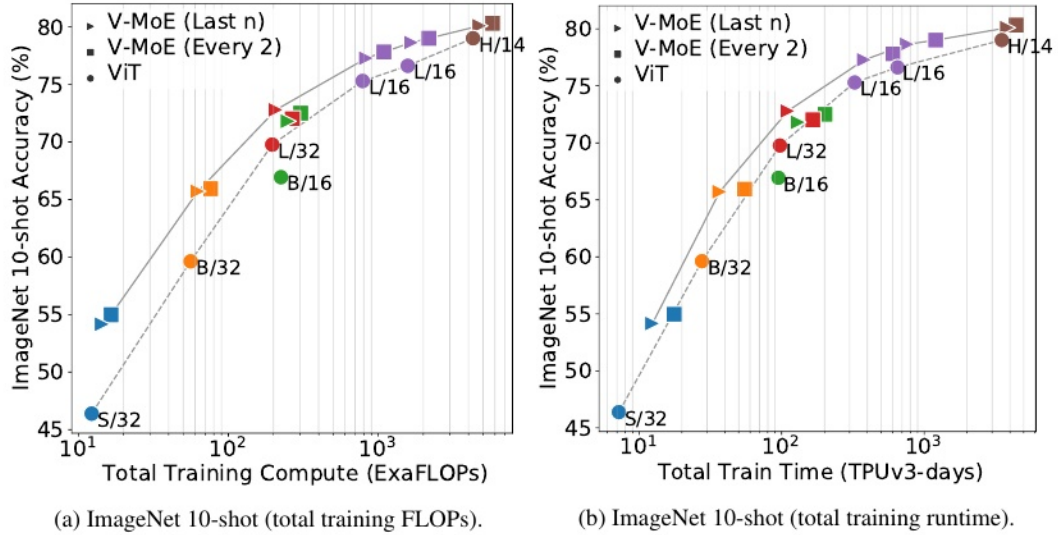


图13:稀疏和密集模型的ImageNet/10shot性能。(a)中的x轴表示训练期间所需的总FLOPs, 而(b)表示相同硬件的总训练时间。

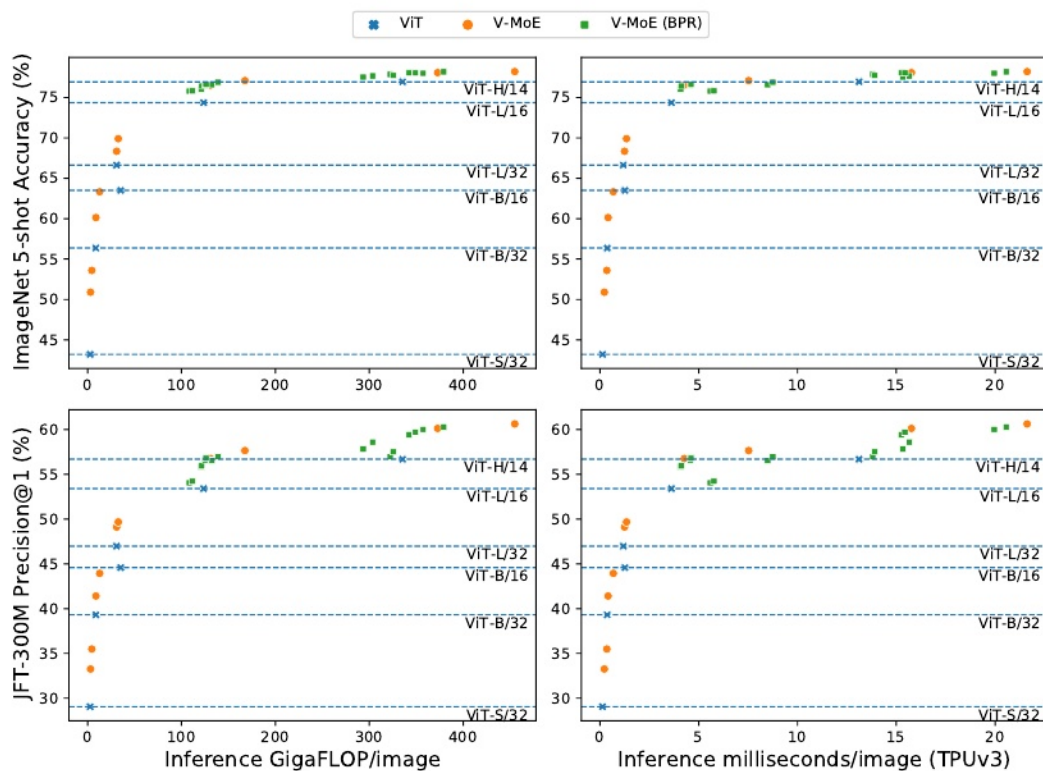


图14:使用优先级路由减少计算。所有模型的性能与推理FLOPs和运行时。具有原始香草路由的V-MoEs用●表示,而**显示的V-MoEs使用BPR和 $C \in \{0.6, 0.7, 0.8\}$ 和 $k \in \{1, 2\}$ 的混合来减少计算。ViT模型如图x所示。图5显示了最大模型的放大版本(与推理FLOPs相比)。

表10:JFT prec@1和ImageNet 5shot的时间和FLOPs不匹配推断结果

Model	Experts	Routing	JFT prec@1	INet/5shot	Time[%]	FLOPs[%]
VIT-H/14	-	-	56.68	76.95	100.00	100.00
VIT-L/16	-	-	53.40	74.36	27.58	36.83
V-MoE-L/16	Last-2	Vanilla	56.76	76.53	32.56	39.02
V-MoE-L/16	Every-2	Vanilla	57.64	77.10	57.40	49.95
V-MoE-H/14	Last-5	Vanilla	60.12	78.08	120.22	111.12
V-MoE-H/14	Every-2	Vanilla	60.62	78.21	164.89	135.59

表11:FLOPs匹配推理结果与批处理优先路由，较低的C，和减少的k。

Model	Experts	At Inference	C	JFT prec@1	INet/5shot	Time[%]	FLOPs[%]
VIT-H/14	-	-	-	56.68	76.95	100.00	100.00
V-MoE-H/14	Last-5	k=2 $\rightarrow$ k=1	1.05	58.60	77.87	111.57	100.26
V-MoE-H/14	Last-5	k=2 $\rightarrow$ k=1	1.25	59.21	77.59	113.67	102.53
V-MoE-H/14	Last-5	k=2	0.5	58.61	77.92	118.14	100.02
V-MoE-H/14	Last-5	k=2	0.6	59.42	78.05	121.68	102.30
V-MoE-H/14	Every-2	k=2 $\rightarrow$ k=1	1.05	59.46	77.82	134.87	100.07
V-MoE-H/14	Every-2	k=2	0.5	59.44	77.70	155.83	100.03

C批处理优先路由

C.1 路由算法

Algorithm 1: Vanilla Routing Allocation

Result: complete assignment of patches to experts (with some potential dropping)

initialize empty buffers with capacity B_e for all experts e (see Section 2);

```
for  $i = 1, \dots, k$  do
  for patch  $p = 1, \dots, N$  do
     $e, w = \text{Router}(\text{TOP} - i \text{ position, patch } p)$ ;
    if  $e$  is not full then
      add patch  $p$  to processing buffer of expert  $e$  with weight  $w$ ;
    else
      skip  $i$ -th expert assignment for patch  $p$ ;
    end
  end
end
end
```

Algorithm 2: Batch Prioritized Routing Allocation

Result: complete assignment of patches to experts (with some potential dropping)

initialize empty buffers with capacity B_e for all experts e (see Section 2);

```
for patch  $p = 1, \dots, N$  do
   $s(p) = \text{ComputeScore}(\text{patch } p, \text{Router}(\cdot))$ ;
end
patch ordering  $\bar{p} = \text{SortPatches}(\text{scores } s, \text{decreasing} = \text{True})$ ;
for  $i = 1, \dots, k$  do
  for patch  $p = (1), \dots, (N)$  according to  $\bar{p}$  do
     $e, w = \text{Router}(\text{TOP} - i \text{ position, patch } p)$ ;
    if  $e$  is not full then
      add patch  $p$  to processing buffer of expert  $e$  with weight  $w$ ;
    else
      skip  $i$ -th expert assignment for patch  $p$ ;
    end
  end
end
end
```

我们探索了几个评分函数，并得出结论，根据每个补丁 p 的最大路由权重进行排序真的很正式， $s(p) = \max_e w_{e,p}$ ，其中 $w_{e,p}$ 是补丁 p 和专家 e 的路由函数 g 的输出(参见4.1节)。我们也尝试了所有TOP- k 权重的总和(而不仅仅是TOP-1)，得到了类似的结果。此外，我们尝试直接学习一个评分函数。在这种情况下，路由器将输出每个patch的 E 个权重(每个专家一个，由softmax函数共同归一化)以及每个patch的分数 $s(p) - 1$ 。我们探索了几个评分函数(线性+ sigmoid等)，得出的结论是，最大路由权重是一个相当好的基线，很难被击败。

该算法的自然扩展包括在补丁专家分配级别进行排序，而不是在全局补丁级别进行排序。与算法2的主要区别在于，排序之后会查看(patch p , TOP- i expert for p) $1 \leq i \leq k$ 的分数。例如，假设 $k = 2$ ，我们有两个补丁， p_1 和 p_2 。假设 p_1 选择路由权重为(0.7,0.2)的专家(e_{11}, e_{12})，而 p_2 选择权重为(0.5,0.4)的专家(e_{21}, e_{22})。在算法2下，尝试patch-expert分配的顺序是:(p_1, e_{11}), (p_2, e_{21}), (p_1, e_{12}), (p_2, e_{22})。然而，如果我们在patch-expert级别使用排序，我们最终会得到:(p_1, e_{11}), (p_2, e_{21}), (p_2, e_{22}), (p_1, e_{12})。后者可能更有意义，因为 p_2 的第二次赋值可能比 p_1 的第二次赋值更相关。然而，我们还没有在经验上尝试过这种方法。

为了完整起见，我们还报告了另一个我们实际实验过的相关算法。我们称之为*skip-patch*。在这种情况下，我们首先设置一个超参数 $S \in (0, 1)$ 。我们将处理patch的一个分数 S ，并直接跳过剩余的 $1-S$ 分数。和之前一样，我们对 N 个patch进行排序

根据某个评分函数 $s(\cdot)$ 。然后，我们直接丢弃最下面 $(1-S)\%$ 的patch，按照算法2中所选的 $M = SN$ patch进行处理。算法3正式描述了这一思路。回到我们前面有两个补丁的例子，如果我们在哪里设置 $S = 0.5$ ，我们将完全丢弃 p_2 ，只处理 $(p_1, e_{11}), (p_1, e_{12})$ 。注意 S 和 C 是两个不同的参数，在给定 S 的情况下调整 C 以避免过多的FLOPs浪费是有意义的。

Algorithm 3: Skip-Patch Routing Allocation

Result: complete assignment of patches to experts (with some **enforced** dropping)

```

let  $S \in (0, 1)$ ;
initialize empty buffers with capacity  $B_e$  for all experts  $e$  (see Section 2);
for patch  $p = 1, \dots, N$  do
  |  $s(p) = \text{ComputeScore}(\text{patch } p, \text{Router}(\cdot))$ ;
end
patch ordering  $\bar{p} = \text{SortPatches}(\text{scores } s, \text{decreasing} = \text{True})$ ;
patch ordering  $\hat{p} = \text{KeepPatches}(\text{TOP} - M, M = SN, \bar{p})$ ;
for  $i = 1, \dots, k$  do
  | for patch  $p = (1), \dots, (M)$  according to  $\hat{p}$  do
    |  $e, w = \text{Router}(\text{TOP} - i \text{ position, patch } p)$ ;
    | if  $e$  is not full then
    |   | add patch  $p$  to processing buffer of expert  $e$  with weight  $w$ ;
    | else
    |   | skip  $i$ -th expert assignment for patch  $p$ ;
    | end
  | end
end

```

C.2 在推理期间应用

上一节介绍的算法的一个吸引人的特性是，它们对模型最初的训练方式是不可知的。实际上，我们首先展示了在使用算法1训练的模型上，通过使用批处理优先路由(算法2)来减少推理时间计算的效果。请注意，两种情况下的模型参数是相同的，包括路由器参数-我们只在推理时应用模型，不涉及进一步的学习-但我们应用了不同的路由策略。总的来说，我们观察到随机丢弃补丁(正如算法1有效地做的那样)会导致性能的急剧下降，当我们只保留一小部分补丁时，正如人们所期望的那样。另一方面，如果我们处理“正确”的补丁——通过算法2——只要我们保留20%左右的补丁，性能就会惊人地健壮。

图15显示了在算法2下， $k = 2$ 的主要every-2专家模型的推理性能作为 C 的函数。我们观察到，随着我们越来越多地约束专家可以处理的补丁数量，性能会缓慢而平稳地下降。

接下来，我们比较了算法1和2的推理性能。V-MoE-H/14的结果见图16, V-MoE-L/16见图17, V-MoE-B/16见图18, V-MoE-S/32见图19。在所有情况下，我们都看到了同样明显的趋势。根据算法1和2的定义，当 $k = 2$ 时，如果 $C \geq 0.5$ ，那么如果路由是平衡的，那么每个补丁得到TOP-1专家处理的几率都有一个不错的变化。因此，这里最有趣的情况是 $C < 0.5$ 。在这种情况下，我们看到算法1和2在性能上的巨大差距，表明选择正确的补丁确实有回报。而且，在大多数情况下，使用15%的patch ($C = 0.15$)就足以匹配密集模型的上游性能。对于少数镜头表示，20%到30%的patch通常就足够了。

总的来说，我们认为算法2提供的灵活性是专家模型的一个相当显著的属性。一旦经过训练，它们就可以在性能和计算之间进行平滑的权衡，而不需要进一步的训练或调整。这在实际设置中当然是有用的，其中用例可以确定可用的资源和手头的约束。

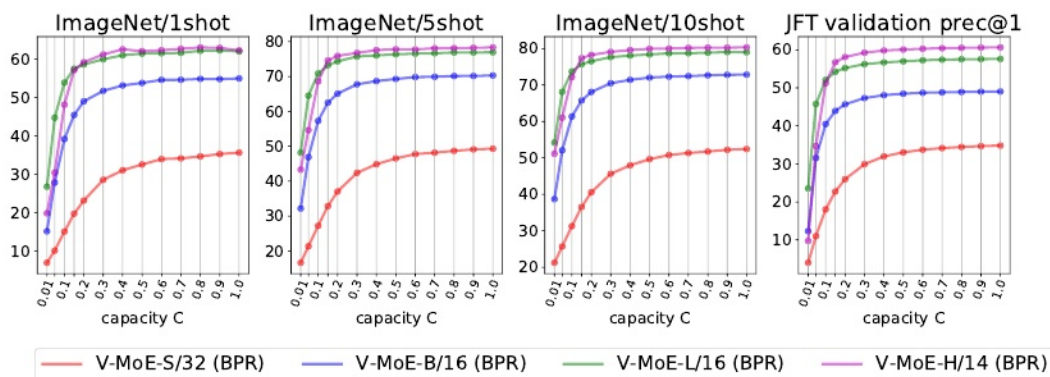


图15:不同容量下 $k = 2$ 的各种every-2 V-MoE模型的推理性能。我们显示了批处理优先路由。

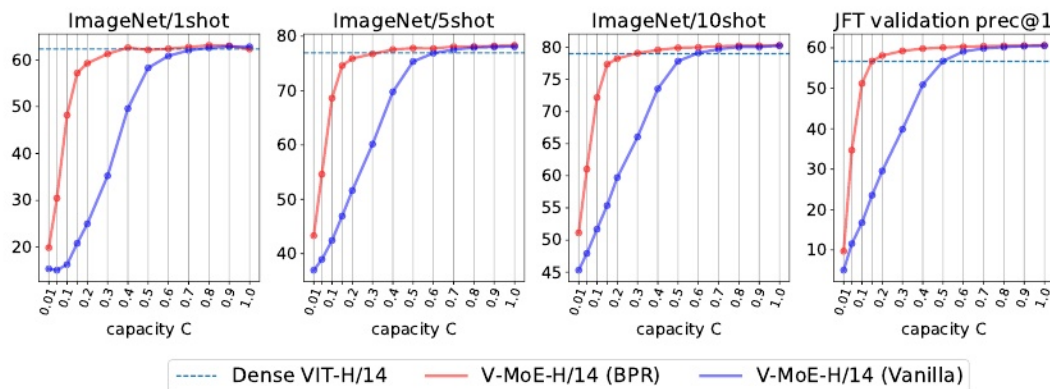


图16:不同容量下 $k = 2$ 的every-2 V-MoE-H/14模型的推理性能。我们展示了批处理优先路由与普通路由。

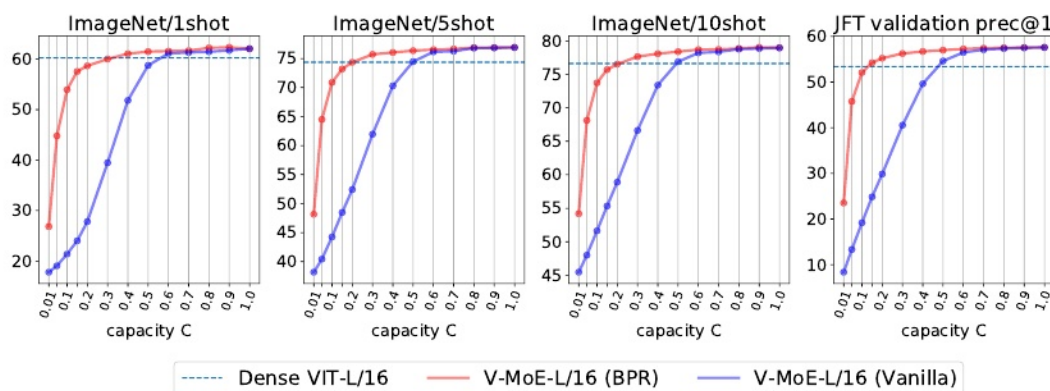


图17:不同容量下 $k = 2$ 的every-2 V-MoE-L/16模型的推理性能。我们展示了批处理优先路由与普通路由。

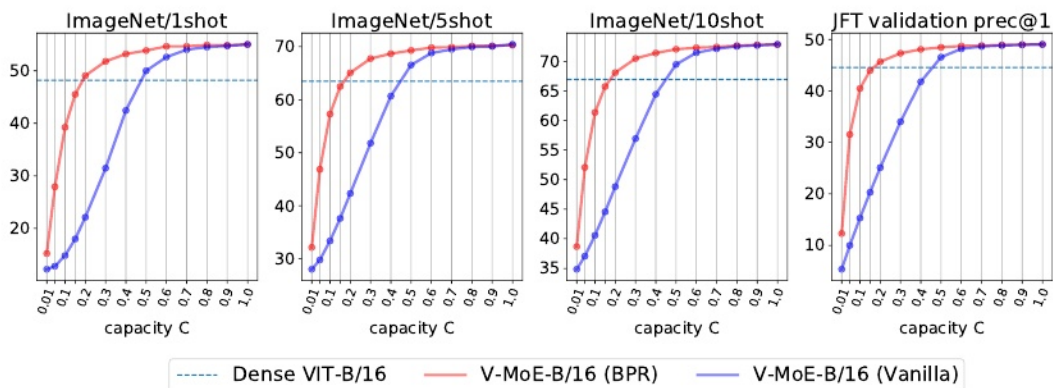


图18:不同容量下 $k=2$ 的every-2 V-MoE-B/16模型的推理性能。我们展示了批处理优先路由与普通路由。

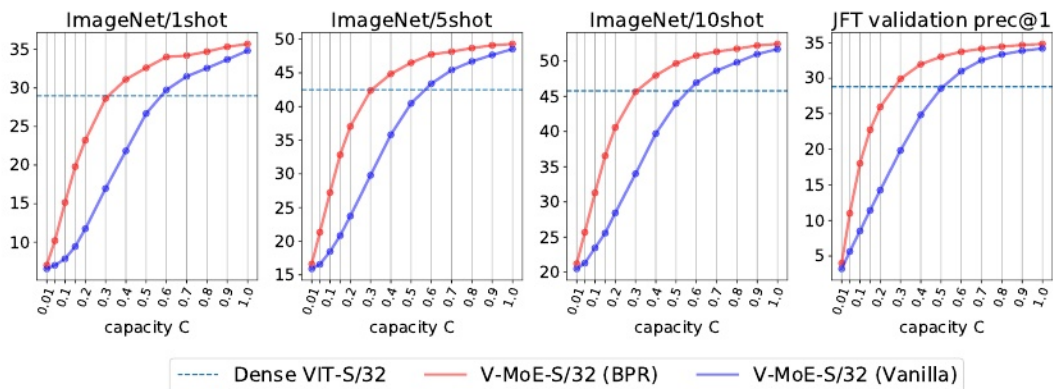


图19:不同容量下 $k=2$ 的every-2 V-MoE-S/32模型的推理性能。我们展示了批处理优先路由与普通路由。

C.3 培训期间应用

前一小节探讨了在预训练模型的推理过程中应用优先级路由。自然扩展包括直接用算法2从头开始训练模型。通过迫使专家使用较小的缓冲区或容量比率(即 $C \ll 1$)，我们可以节省大量的训练FLOPs，同时仍然有望获得相对于密集模型的体面的性能改进。

我们展示了三种模型的结果:V-MoE-S/32、V-MoE-B/32和V-MoE-L/32。为了完整性，我们比较了算法1和算法2。在所有情况下，当我们使用算法2进行训练时，我们都看到了很大的改进。然而，当我们使用满容量($C \geq 1.0$)时，我们期望两种算法的行为方式相当相似，因为只要路由合理平衡，就不需要下降。

图20和21分别为 $k = 1$ 和 $k = 2$ 时的V-MoE-S/32。在这两种情况下，我们都能够以大约80%的训练FLOPs匹配密集的上游性能。此外，在每种情况下，大约85%和80%的训练FLOPs足以匹配少量射击评估性能。总的来说，在训练像V-MoE-S/32这样的小型模型时，我们可以节省20%的FLOPs。

图24和图23分别为 $k = 1$ 和 $k = 2$ 时的V-MoE-B/32。同样，专家模型最多有80%的训练FLOPs与密集模型的上游性能相匹配。此外，我们可以节省大约10%的训练FLOPs，同时保持或提高少镜头表示质量。

最后，图24和图25给出了 $k = 1$ 和 $k = 2$ 时viti - 1/32的结果。值得注意的是，70%到75%的训练FLOPs足以模拟上游密集性能。请注意，当 $k = 2$ 时，最低容量($C = 0.1$)的性能已经超过了密集的上游精度。专家模型还能够提供相同的少数射击性能，同时节省超过20%的训练FLOPs。

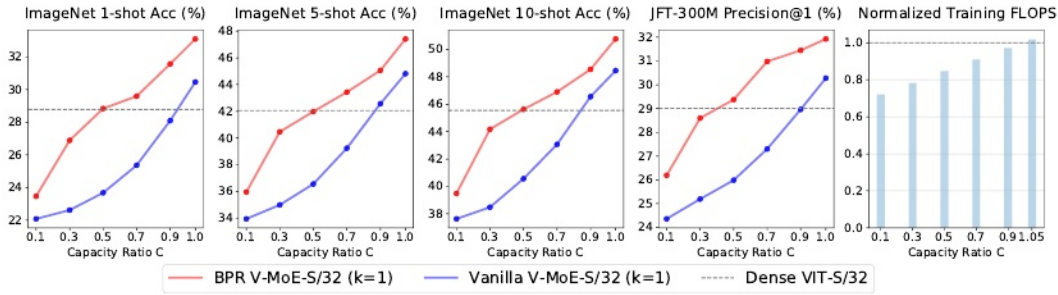


图20:使用批处理优先路由进行训练。型号:V-MoE-S/32, $k = 1$ 。平均4粒以上。

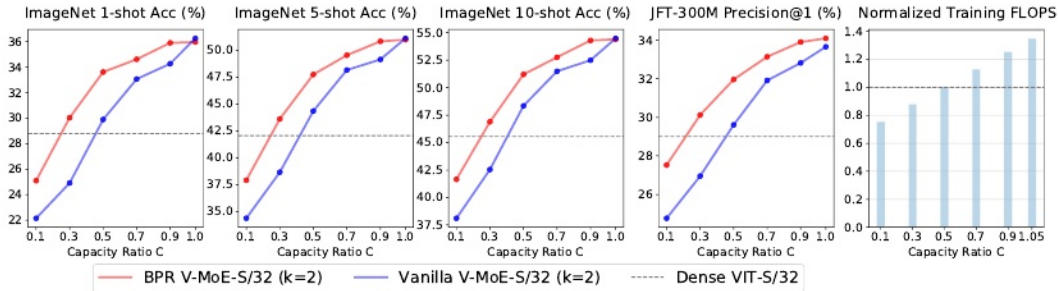


图21:使用批处理优先路由进行训练。型号:V-MoE-S/32, $k = 2$ 。平均4粒以上。

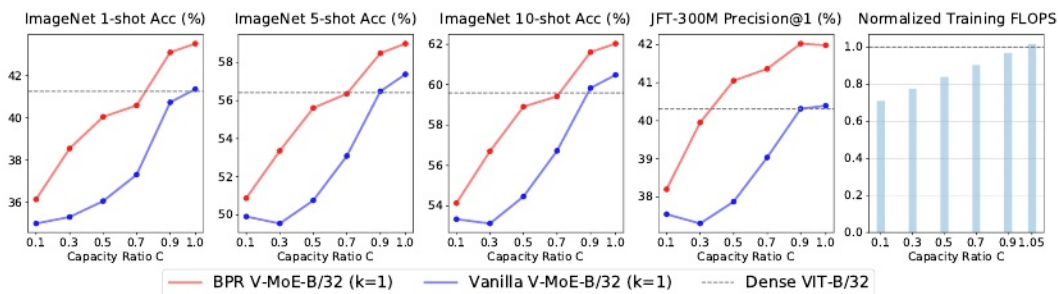


图22:使用批处理优先路由进行训练。型号:V-MoE-B/32, $k = 1$ 。平均4粒以上。

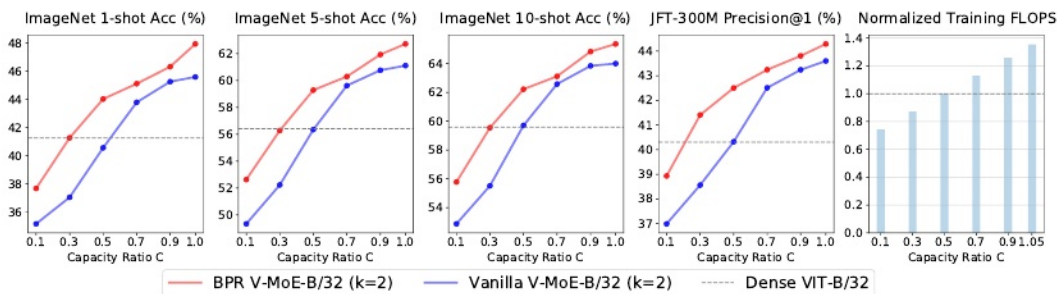


图23:使用批处理优先路由进行训练。型号:V-MoE-B/32, $k = 2$ 。平均4粒以上。

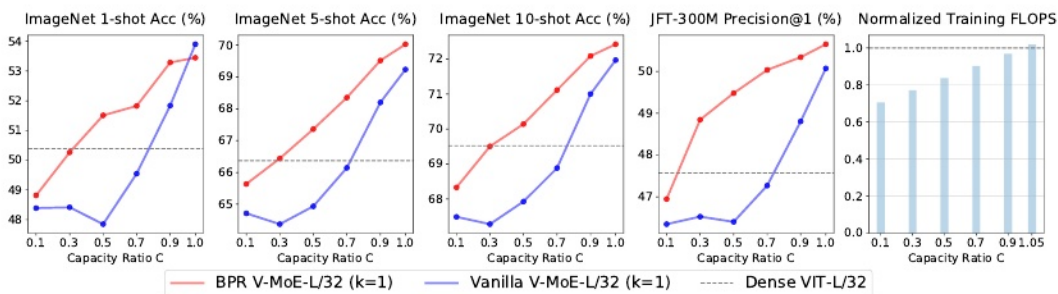


图24:使用批处理优先路由进行训练。型号:V-MoE-L/32, $k = 1$ 。平均4粒以上。

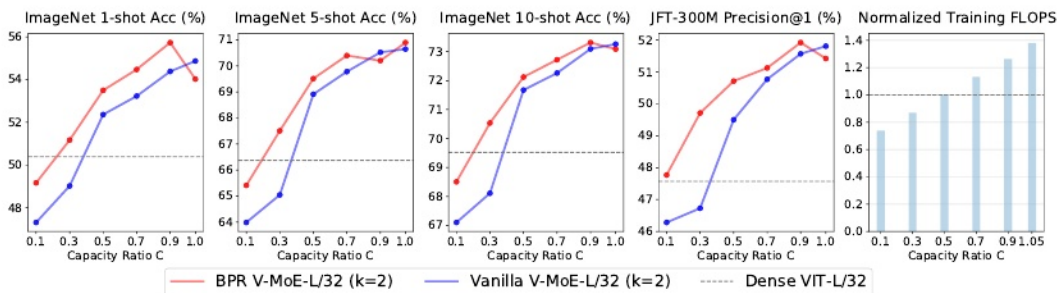


图25:使用批处理优先路由进行训练。型号:V-MoE-L/32, $k = 2$ 。平均4粒以上。

C.4 微调期间应用

我们还研究了在微调中使用max-routing算法的效果。我们考虑V-MoE-S/32模型在各种容量下进行预训练，包括使用和不使用批处理优先路由。我们在ImageNet上对它们进行微调，以查看优先级路由在下游微调 and 推理期间的效果。如图26所示。

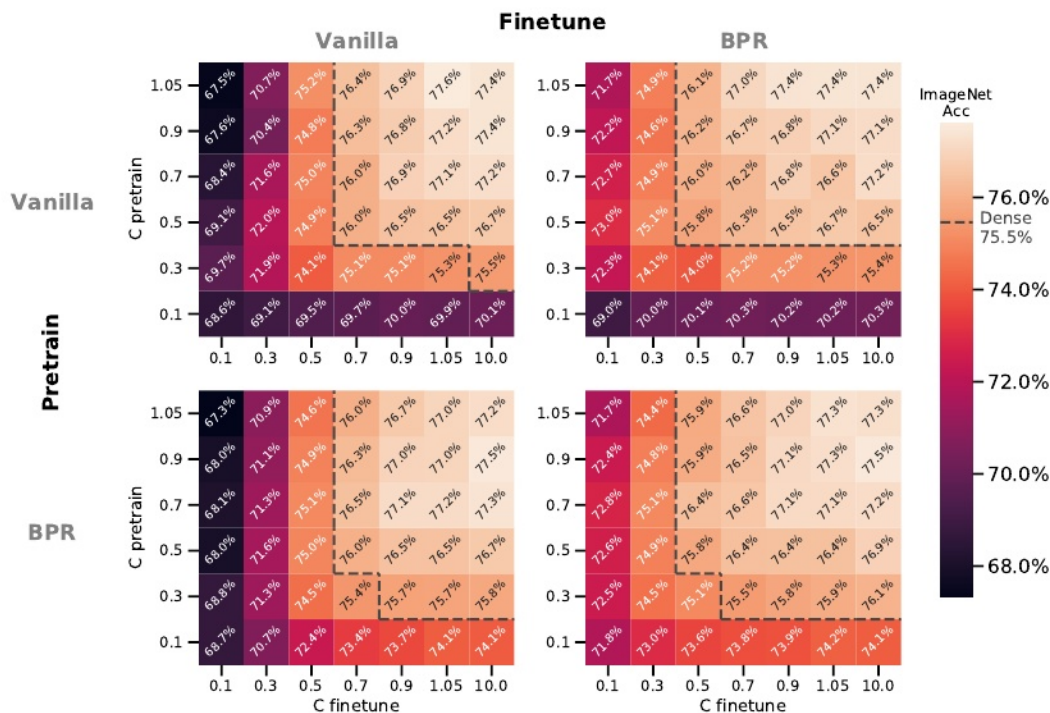
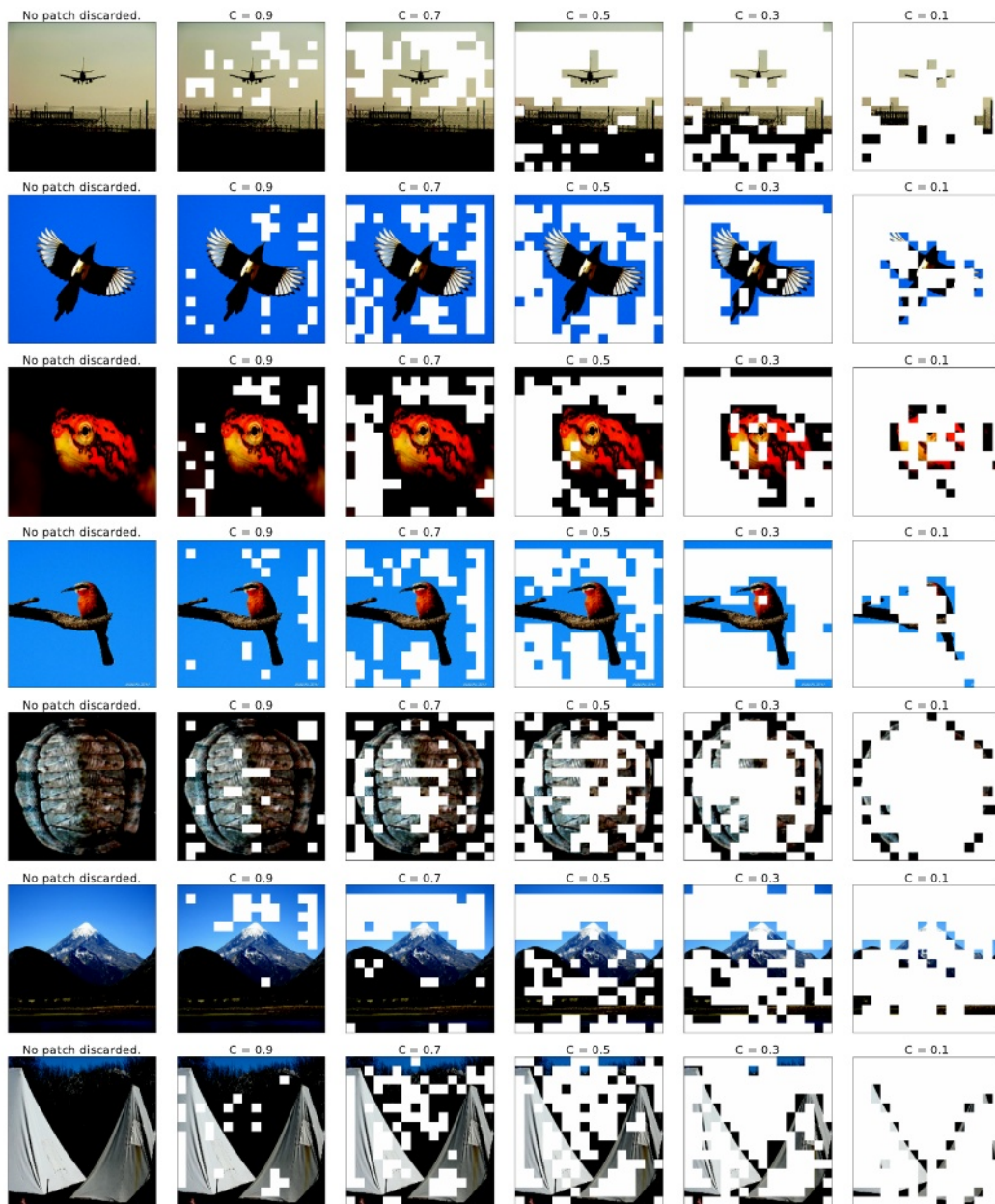


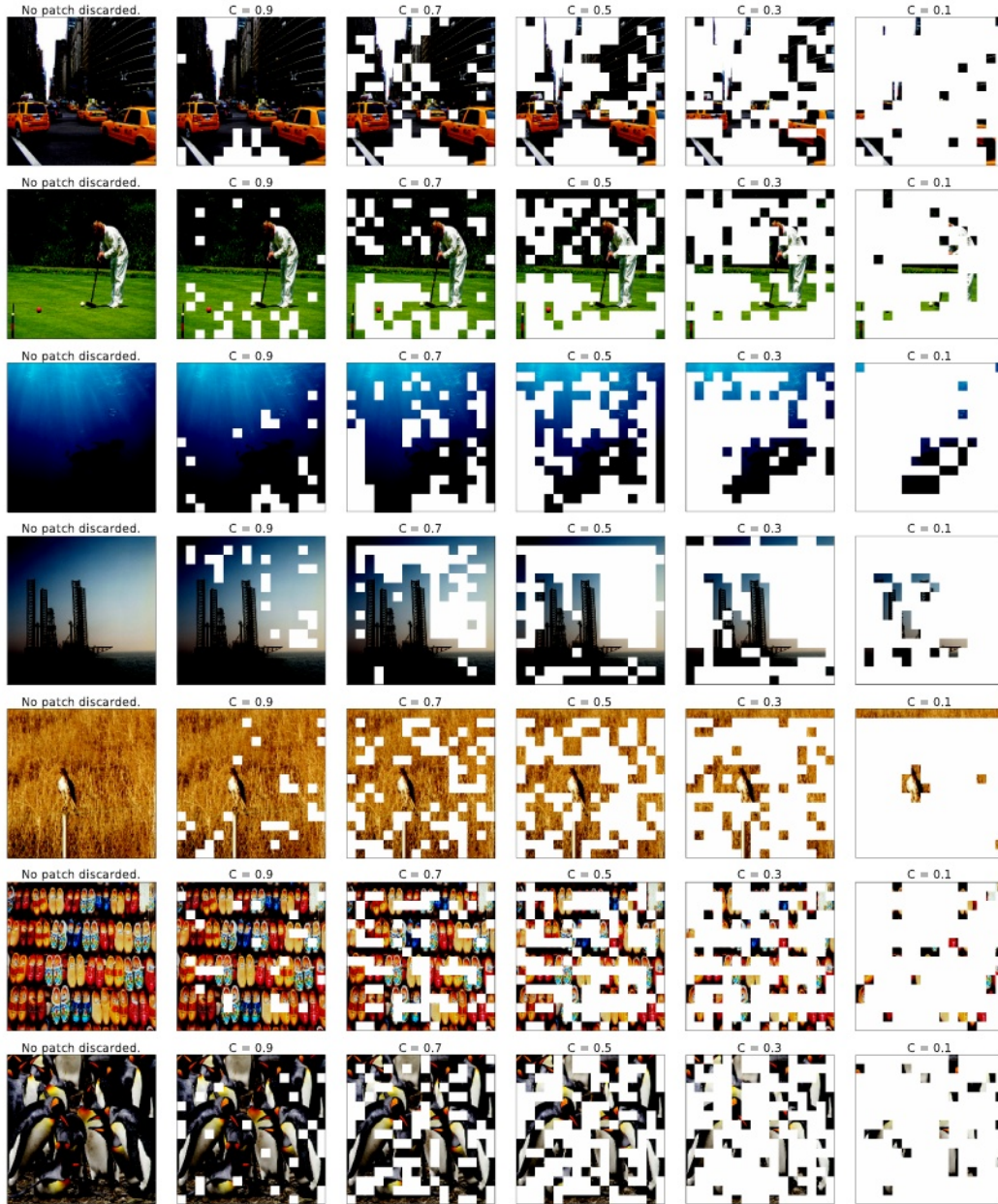
图26:使用批处理优先路由进行微调。型号:V-MoE-S/32, k = 2。

可以得到几个结论:

- 下游微调结果受到产能的显著影响，产能降低至0.1，精度从77.4%降至68.6%。
- 批处理优先化路由可以恢复部分性能下降;如果在预训练和微调期间应用，在相同容量下，准确率提高到71.8%。
- 在微调期间保持高容量比在预训练时更重要。例如，在下游和上游同时应用优先级路由时，预训练期间C = 1.05，微调期间C = 0.1，准确率为71.7%，但反之则明显更好，准确率为74.1%。在这两种情况下，优先级路由都是改善微调和预训练期间低容量影响的关键。

D补丁掉落的例子





E模型分析

之前的一些作品提出了基于专家混合的深度模型;他们中的大多数也提出了有希望的结果。不幸的是, 尽管目前对这组技术感到兴奋, 但对于这些复杂模型的内部工作原理确实知之甚少。探索性实验揭示了路由器和专家专业化的机制, 可以为新的算法提供信息。我们试图在这里提供第一个这样的分析, 我们实际上发现这对开发论文中提出的一些算法很有用。

E.1 路由器的价值

训练完一个稀疏模型后, 最自然的问题是, 学习到的路由器是否在做一些有用的事情。有几种可能出现问题的方式。例如, 如果专家最终实现非常相似的功能, 路由器可能会成为负载均衡器。或者, 路由器可能只是简单地选择次优分配。作为第一个测试, 我们一次用一个均匀随机的路由器替换一个路由器。为此, 我们采用预训练模型, 特别是 $k = 2$ -的V-MoE-L/16, 并在干扰路由器时重新评估其上游和少射性能。图27包含了结果。我们用红色表示了预训练模型的原始性能, 即应用所有学习到的路由器时的性能。我们还展示了用一个均匀随机的路由器独立地、隔离地替换每个路由器的影响——层ID显示在x轴上。特别是, 新路由器以白高斯方式对权重进行采样, 因此对于任何给定的输入, 每对专家都同样有可能成为TOP-k选择。我们还尝试随机排列输出权重——以避免应用路由权重的分布移位——这使结果变得更糟。

总的来说, 我们观察到最后两层——21层和23层——为上游模型的工作提供了必要的路由(JFT中的验证精度为1)。我们在其他模型中也看到了类似的模式。有趣的是, 在最前面的MoE层(本例中是第21层)中, 获得正确的路由是最重要的。该模型对大多数中间层的错误路由具有鲁棒性——第9层在这里是个例外。这一观察结果促使我们尝试仅在最后(例如21和23)使用MoE层训练稀疏模型, 并获得了出色的结果(并节省了计算量)。

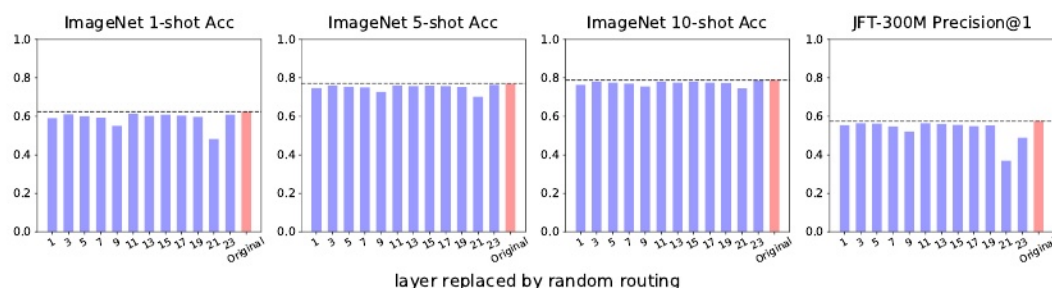


图27:V-MoE-L/16随机更换一层路由器

在分析了图27的结果之后, 一个自然的后续问题是, 该模型对复合错路由是否具有鲁棒性?我们通过观察当我们用均匀随机的路由器替换多个连续的MoE层时会发生什么来回答这个问题。图28显示了结果。我们从最下面的MoE层开始, 对于网络中的每一个MoE层 i , 我们评估1到 i 层(都包括在内)的路由器随机行为的模型。不幸的是, 在这种情况下, 性能会像人们预期的那样迅速下降。令牌在一定程度上遵循随机游走(如果我们忽略容量问题), 然后使用正确的剩余路由器。如果随机漫步足够长, 性能就会严重下降。我们得出结论, 训练模型中的令牌路径远不是随机的或无意义的。

E.2 专业专家

在图29中, 我们展示了具有24个MoE层的大型模型的结果, 每个层有32个专家。在JFT上训练模型并在ImageNet上对其进行微调之后, 我们对ImageNet图像进行了前向传递(直到预logits)。按递增顺序, 每个图对应一个MoE层的。

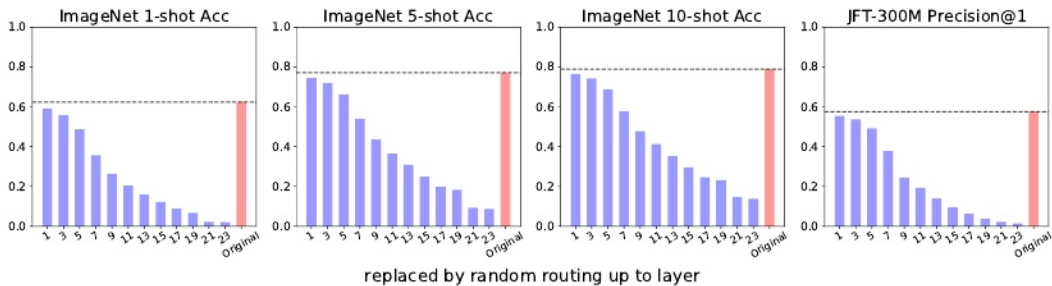


图28:将V-MoE-L/16的所有层替换为随机路由器。

x轴对应每层32个专家，y轴是1000个ImageNet类(按不同的调整顺序每个图;即 $i \neq j$ 时，第 i 层和第 j 层中的第5类一般是不同的)。对于每一对(专家 e ，类 i)，我们显示了该特定专家 e 的所有类 i 图像对应的补丁的平均路由权重。直观地说，这是 i 类图像激活和使用专家 e 的数量的代理。图29显示了最后几层的强专家-类相关性。换句话说，似乎专家专门区分一小部分类(那些主要通过专家路由的类)。然而，在最初的MoE层中，我们没有观察到这种相关性，我们得出结论，专家可能会关注补丁的不同方面，这些方面可能对所有类别(背景，基本形状等)都是共同的。

为了进一步研究第一层专家背后的逻辑，图30显示了所选专家与补丁id或位置之间的相关性。模型将每个图像划分为相同数量的补丁——假设补丁大小为 14×14 ，图像大小为 $224 \times 224 \times 3$ ，则有256个补丁(有时我们会添加一个额外的可学习令牌)。我们给每个patch添加一个位置嵌入，帮助模型跟踪相对排序。在这种情况下，我们看到，对于前几个MoE层，路由器倾向于根据他们的补丁id向专家分发补丁。一个简单的解释可能是，相似位置的补丁通常共享一个专家学会处理的视觉特征——比如，图像的角落、背景或带有物体的图像中心。

E.3 路由权值分布

大多数关键模型超参数，如处理每个补丁 k 的专家数量或控制补丁掉落量的专家缓冲容量比率 C ，都可以按层调整。例如，如果我们在较低的层中没有看到专家专门化，我们可以简单地在那里设置 $k = 1$ ，以避免浪费计算。然而，在最后几层增加 k 以允许概念的可组合性可能是有用的，比如当我们试图识别几个对象时。图31显示了 $k = 2$ 的稀疏模型的TOP-1和TOP-2权重分布。可以得出两个主要结论。首先，在较低的层次中，两种选择似乎具有相似的大小——因此，两者确实都有助于组合表示。此外，这些层的权重通常很低——注 $1/E \approx 0.03$ 是可以分配给最上面选择的专家的最小权重——这可以解释为路由器在专家中有些冷漠。其次，当补丁向网络末端移动时，趋势明显发生变化。其中，TOP-1和TOP-2的权重分布分化强烈，前者接近1.0，后者接近0。这意味着网络顶部的固有 k 更接近于1(而不是实际的 $k = 2$)。我们之前提到的可组合性在补丁级别上可能确实不需要，因为补丁对于大型网络来说是相当小的(这里是 14×14)，并且可能很难识别几个概念。尽管如此，图31中所示分布的一些尾部仍然在最后一层使用了两个专家。

我们想指出的是，每个图像都受到通过其补丁的大量路由决策的影响。具体来说，图32显示了大多数图像如何通过池化所有补丁来使用-on聚合-每层中的大多数专家。这促使我们努力尝试通过丢弃或不处理对最终分类无用的补丁来节省计算。我们将在第4节中详细介绍这一点。

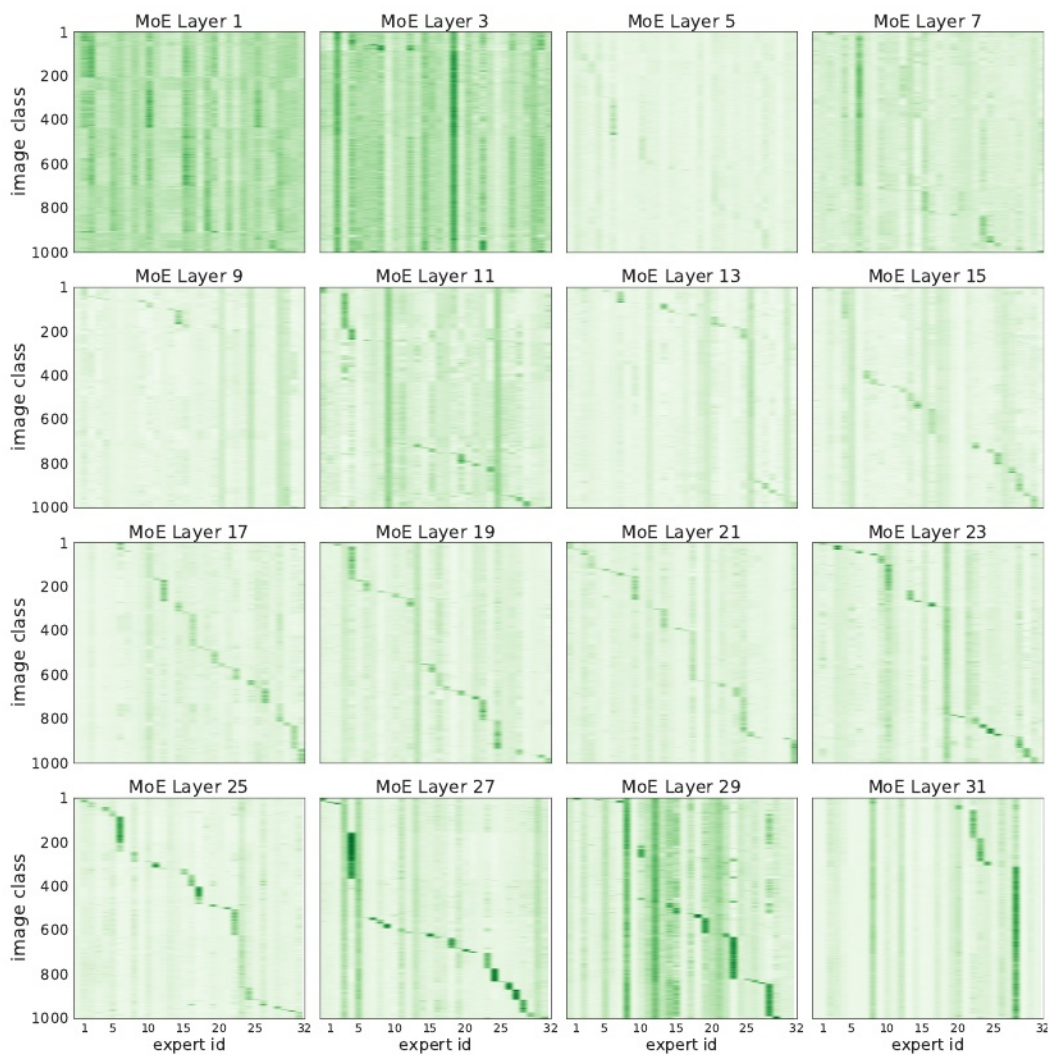


图29:每个类别所选专家的平均权重。我们展示了每2个V-MoE-H/14的16个MoE层。x轴对应于一层中的32位专家。y轴是1000个ImageNet类;两个轴的排序在不同的图上是不同的。对于每一对(专家e, 类i), 我们显示了该特定专家e的所有类i图像对应的补丁的平均路由权重。

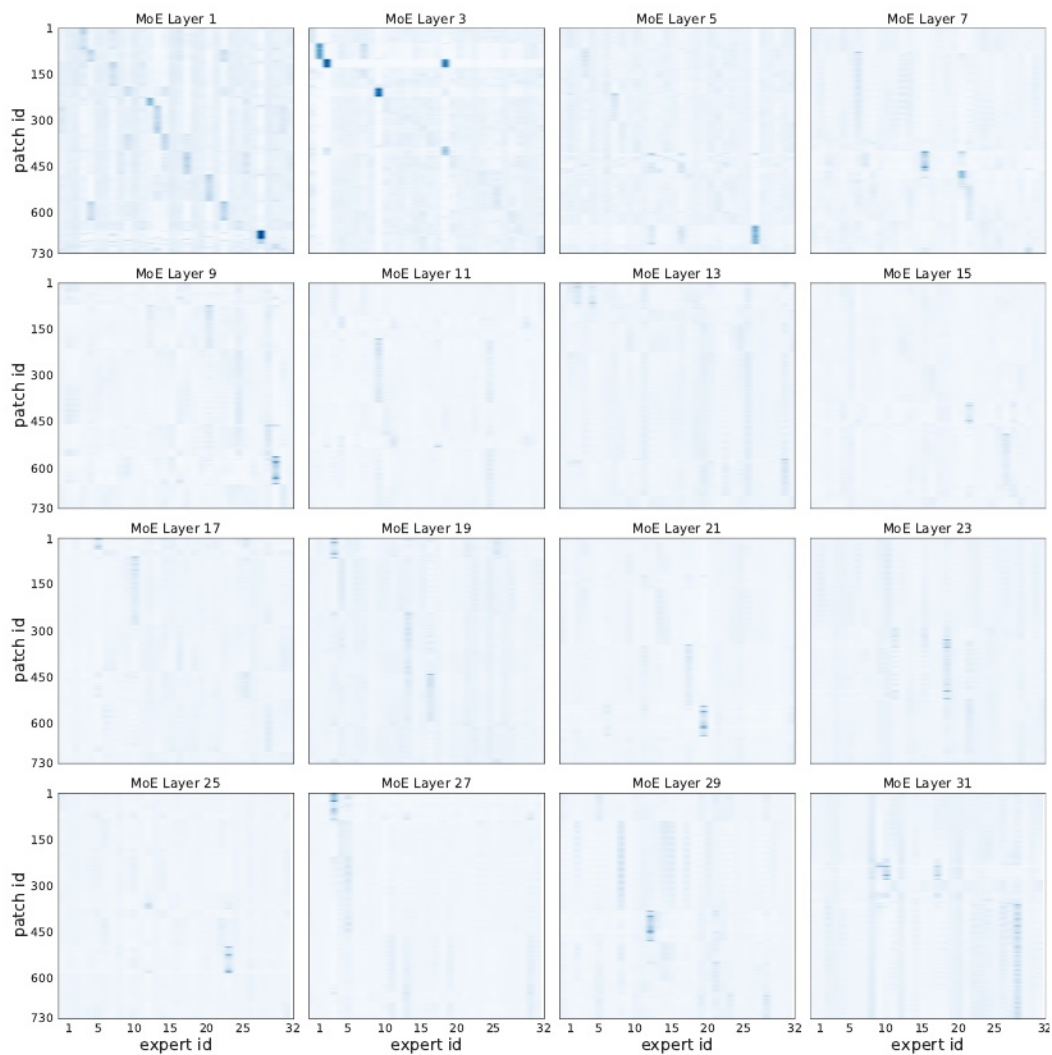


图30:在每2个V-MoE-H/14微调模型上, 选定专家每个补丁位置的平均权重。x轴对应一层的32位专家。y轴为ImageNet图像中的730个补丁, 尺寸为14x14, 分辨率为(384,384,3);x轴的排序在不同的地块上是不同的。对于每一对(专家e, patch-id i), 我们显示了分配给该特定专家e的所有patch-id i的平均路由权重。

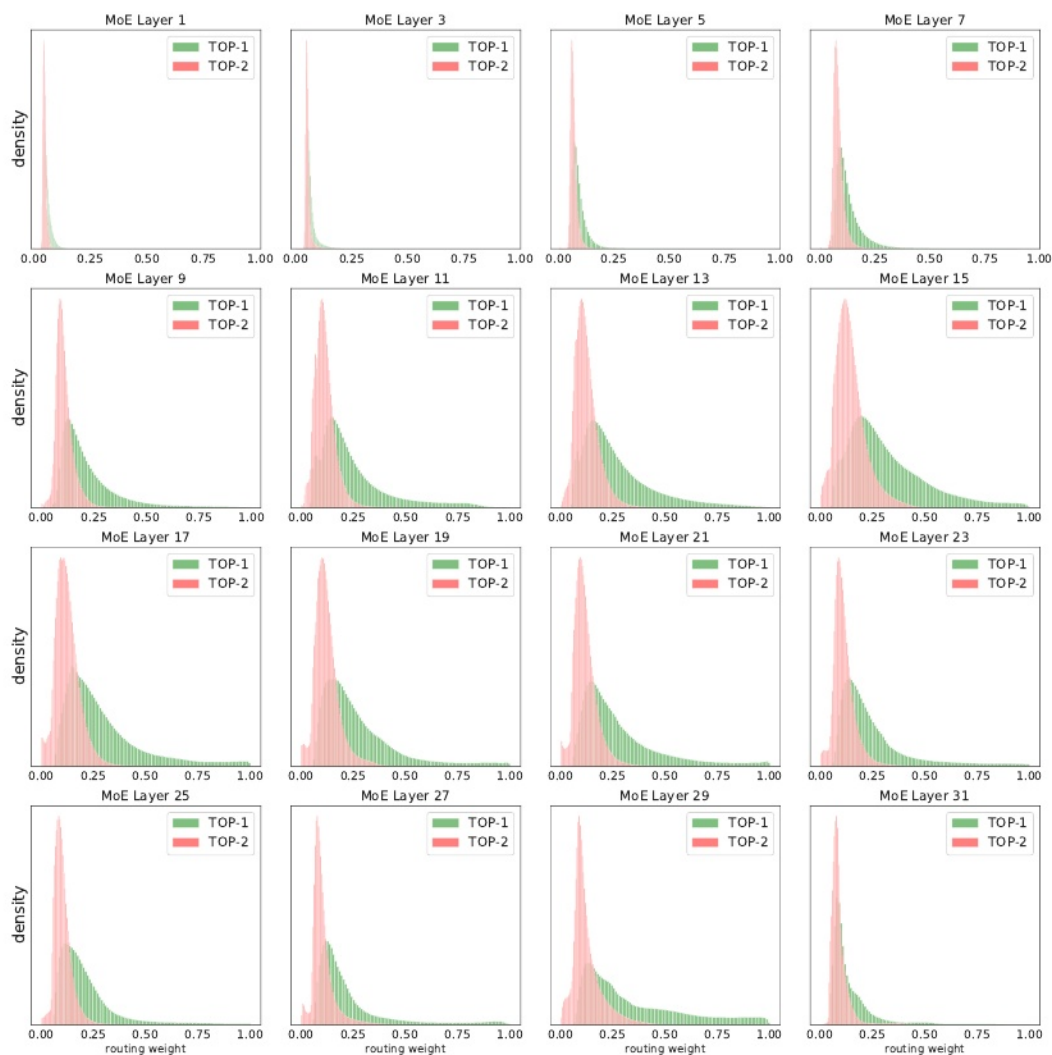


图31: TOP-1和TOP-2所选专家的路由权重分布。我们展示了在ImageNet上微调的V-MoE-H/14模型的TOP-1(绿色)和TOP-2(红色)权重的分布。注意，对于任何给定的补丁，这些权重不需要加到1上，事实上它们不会加到1上，因为我们在TOP-k选择之前应用softmax。

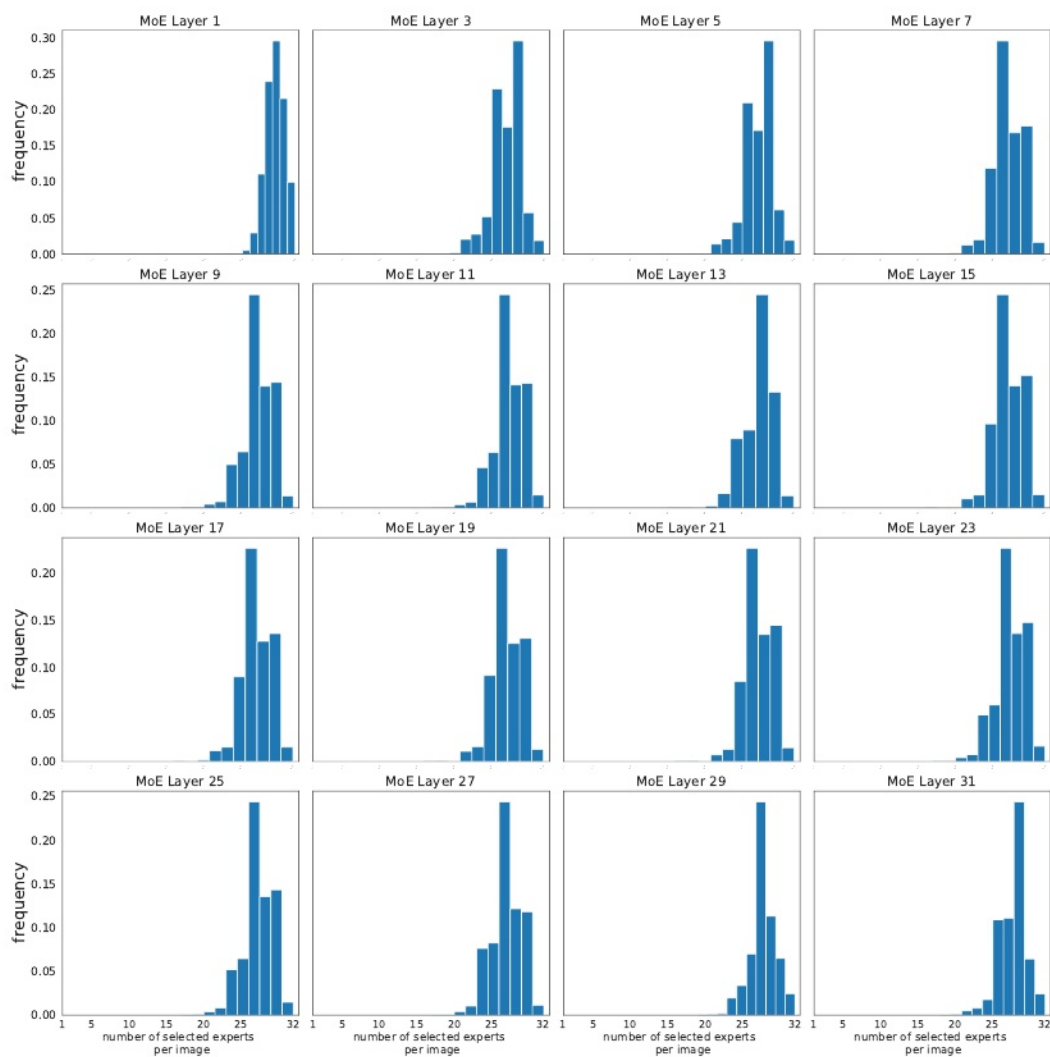


图32:每张图像选定的专家数量(在汇集所有补丁的选择后)。我们展示了在ImageNet上微调的V-MoE-H/14模型的每层每幅图像的使用专家总数分布。在这种情况下，每张图像有730个补丁。即使大多数专家至少被选中一次——这就是我们在这里绘制的——我们希望一些专家被图像的补丁更频繁地选中，并且具有更高的平均权重。

E.4 在推断时改变k

我们现在探索专家模型的一个显著方面:它们的灵活性。有些令人惊讶的是,我们已经观察到稀疏模型对训练和推理配置之间的不匹配具有相当强的鲁棒性。在本节中,我们将探讨使用某些原始k进行训练的效果,同时在不同 $k' \neq k$ 的推理时应用该模型。这可以方便地控制(减少或增加)特定生产系统中每个输入的FLOPs数量。

图33是基于 $k = 1$ 训练的V-MoE-S/32模型。我们在推断时间对一系列新的k's评估上游和少射指标。请注意,我们在任何情况下都不会执行任何进一步的训练,并且在所有情况下模型参数(包括路由器)都是相同的。唯一的区别是我们对每个输入应用的专家数量,我们激活的网络的数量。红色表示原始模型的表现,蓝色表示新模型的表现。最后,对于每个k',我们用黄色表示最初用 $k = k'$ 训练的V-MoE-S/32模型的性能,正如我们预期的那样,在k'中增加。我们看到,在推理时间将k的值从其原始值($k = 1$)增加一个或两个单位,实际上可以显着提高性能,无论是上游还是few-shot。然而,在某些时候,如果新的k's太大,性能就会开始受到影响——可能是因为模型没有为线性组合中应用的总输出路由权重的新分布做好准备,并且给定输入的次优专家开始为其表示做出贡献。

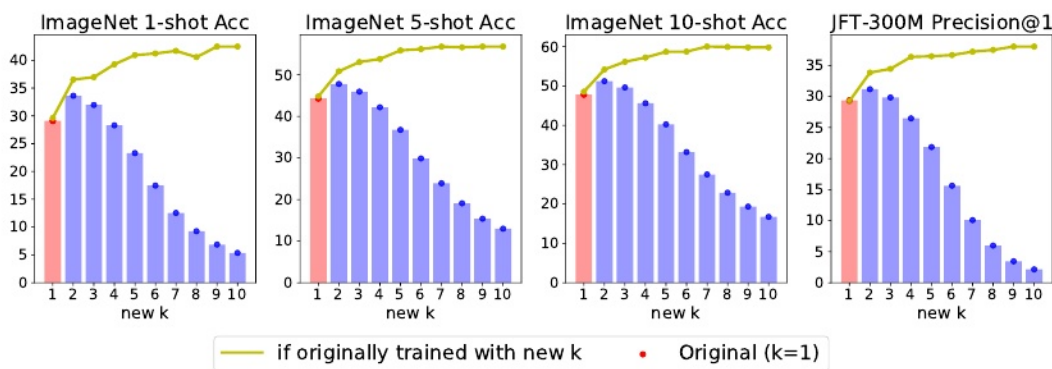


图33:初始V-MoE-S/32 every-2模型以 $k = 1$ 进行训练。

图34显示了原始型号为 $k = 2$ 的V-MoE-S/32的情况。趋势有些相似。通过应用 $k' = 3$ 或 $k' = 4$,我们获得了适度的改进,而通过将k降低到 $k' = 1$,我们获得的性能非常类似于直接使用 $k = 1$ 训练的模型的性能,特别是对于few-shot。这很有趣,因为我们可以通过预先设置 $k = 2$ 来为训练投入更多的FLOPs,同时推迟推理k的选择,而不会损失潜在的性能。我们在第4节中进一步探讨了这些想法。此外,在这种情况下,大k值的性能下降不那么严重,可能是因为训练的模型被用来组合几个不同的专家(图33不是这种情况)。

最后,在图35中,我们给出了用 $k = 5$ 训练上游模型的情况。这是一个训练成本很高的模型,我们看到我们可以将k的推理值从 $k' = 3$ 更改为 $k' = 7$,结果与其最优值相似,如果我们一开始就使用这些值进行训练。在这一点上,模型足够稳定,可以处理大的k值,但当我们设置 $k' = 1$ 时,它会受到更多的影响,因为模型不习惯选择单个专家,而且我们怀疑TOP-1专家可能对这个模型没有那么大的重要性或权重,因为在训练时每个输入选择了五个专家。当然,这可能不仅仅是路由权重分布的问题。当训练 $k = 1$ 时,专家本身可能会有很大的不同,比如,更独立,而当训练 $k = 5$ 时,可能会有更多的团队成员。

E.5 在微调过程中改变k

我们还考虑了微调和推理期间调整所选专家数量的影响。我们考虑前面提到的V-MoE-S/32模型,有32位专家,预先训练与